SESSION 1

Introduction to Context Engineering

From prompts to the systems that control what information reaches the model.

How do you get good outputs from LLMs?

`f(a, b, c, …) = good output`

Good outputs are a function of what?

Attention is a relevance map

> *"Given token X, which other tokens are most relevant to understanding it?"*

Every token competes with every other token for the model's attention budget.

More context = more competing candidates.

The wall every developer hits

Brilliant model. Real problem. Then:

- **No private knowledge** — your docs, codebase, users
- **No memory** — yesterday's chat is gone
- **Confident hallucinations** — fabricates when it doesn't know

Context engineering is how you fix all three.

What is Context Engineering?

The discipline of designing the architecture that feeds an LLM the right information at the right time.

> *Context engineering = **entropy reduction**. High-entropy human intent → low-entropy representations a machine can process.*

Prompt engineering is a subset.

Prompt engineering = crafting one message. Context engineering = orchestrating the entire information system around the model.

The context window is RAM

CPU

The LLM. The reasoning engine.

RAM

The context window. Active working memory.

DISK

Vector DBs, files, APIs. External storage.

Once RAM fills, older information gets evicted. Important details can vanish.

Bigger windows ≠ better

100K, 200K, 1M+ tokens available — and yet:

- **Accuracy drops** as irrelevant info crowds signal
- **Hallucinations rise** when grounding is buried
- **Costs climb** linearly; quality climbs sub-linearly

Performance degrades far before the window fills.

Context Rot

Information goes stale — and the agent can't tell.

The system that worked at turn 1 is wrong by turn 50. Facts age. Tool outputs grow obsolete. The window still references them as if fresh.

Context hygiene = ongoing maintenance, not one-time setup.

03 / THE FOUR OPERATIONS

Write · Select · Compress · Isolate

Every context engineering strategy maps to one of these four levers.

W / S / C / I — in one slide

WRITE

Create context that doesn't exist yet. Plans, scratchpads, learned memory.

SELECT

Pull the right context per step. Retrieval, agentic search, tool loadout.

COMPRESS

Shrink without losing signal. Summarize, prune, offload.

ISOLATE

Keep concerns separate. Sub-agents, sandboxes, state schemas.

Context prioritization

When the budget is tight:

- **Critical** — current code, error messages, active task → always
- **Helpful** — project structure, conventions → if space permits
- **Optional** — historical decisions, background → only if needed

*Rule: models can ignore irrelevant info, but cannot perform without relevant info. When in doubt, **prioritize recall**.*

"Find the smallest set of high-signal tokens that maximize the likelihood of your desired outcome."

— Anthropic, Effective Context Engineering for AI Agents

Key takeaways

1. **Context engineering is entropy reduction.**
   Turn high-entropy human intent into low-entropy machine-ready input.

2. **Context is the RAM.**
   Everything the model sees at inference: system, history, tools, retrieved info.

3. **Four levers: Write, Select, Compress, Isolate.**
   Every technique you'll see in this course maps to one of these.

UP NEXT: SESSION 2

Engineering Context in Agentic Systems

From naive agent loops to production-grade context management.

SESSION 2

Engineering Context in Agentic Systems

From naive agent loops to production-grade context management.

Chat → Agent: what changes

CHAT MODEL

Human message is the primary input. Context is manually crafted by the user. Single turn or short conversation.

AGENT

Context flows from **tool calls** during the trajectory. Machine-generated, accumulating. Dozens to hundreds of tool calls.

500K

tokens / run
(naive deep research)

$1–2

per single
agent run

50–100

tool calls in
production agents

The naive agent doesn't scale. Context engineering is the fix.

The naive agent loop

```
USER QUERY → LLM THINKS → TOOL CALL → RESULT → (loop)
```

Each iteration adds the full tool result to message history. **Context grows monotonically.** The agent can't start over — it needs continuity across steps.

Token-heavy results (web search, file reads) + sheer volume of iterations = the core engineering problem.

"Find the smallest set of high-signal tokens that maximize the likelihood of your desired outcome."

— Anthropic · the engineering challenge, restated for agents

Write: scratchpads, memories, filesystem

- **Scratchpads** persist *during* a task — todo.md, plan files, JSON progress logs
- **Memories** persist *across* sessions — Cursor/Windsurf auto-generate them
- **Filesystem** = a single interface for storing infinite context outside the window

Agents read and write files as external memory.

Select: agentic retrieval beats brute force

- **Vector store** — classic chunking + embeddings (high infrastructure)
- **Agentic search** — `llm.txt` + a fetch tool (Lance Martin: "extremely effective")
- **Context stuffing** — feed all 3M tokens (expensive, degrades quality)

Claude Code uses grep/glob. No indexing. Just progressive disclosure.

"Claude Code doesn't do any indexing. It's just doing agentic retrieval — using simple tool calls, no indexing whatsoever, and it obviously works extremely well."

— Lance Martin

Compress: offload before you summarize

1. Execute tool call
2. **Write full result to disk**
3. Return **summary + pointer**
4. Agent re-fetches only when needed

Summarization is lossy. Offload first as a safety net.

Compression toolkit

SUMMARIZE

With **high recall**. Exhaustive bullets so the agent knows whether to re-fetch.

AUTO-COMPACT

Claude Code triggers at **95% capacity**. Don't wait for OOM.

TRIM

Clear verbose tool outputs after they've been processed.

CACHE

Cache invariant context — system prompts, tool defs. **Don't reprocess.**

Isolate: sub-agents & sandboxes

- Each sub-agent gets a **clean window** — explores 10Ks of tokens, returns 1–2K summary
- **Sandbox** persists state across turns — houses images, audio, data files the LLM never sees
- **State objects** (Pydantic models) — typed buckets; selectively expose fields per stage

Expands total system capacity without bloating any single window.

Read tasks parallelize. Write tasks don't.

READ TASKS ✓

Sub-agents collect info in parallel; **one** writer synthesizes at the end.

Example: Anthropic's deep researcher.

WRITE TASKS ×

Parallel writers make **contradictory choices** that compound.

Example: Flappy Bird sub-agents built mismatched Super Mario assets.

Don't build multi-agents

> *Cognition (makers of Devin) argues multi-agent frameworks create **fragile systems** through dispersed decision-making.*

Parallel agents make unstated assumptions that conflict, producing incoherent output.

The real frontier isn't dispersing work — it's improving how single agents manage context.

Inter-agent boundaries are where it breaks

When isolated agents must share context, the **boundary protocol matters enormously**.

- **Shared plan files** for serial long-running tasks
- **Boundary compression** — Cognition uses fine-tuned models specifically for this
- **Full trace sharing** > individual messages — single messages lose decision context

The core engineering challenge

LEAN

Remove everything the agent doesn't need for the next decision.

COMPLETE

Preserve all info critical for the current task.

FRESH

Prioritize recent, validated context over stale history.

STRUCTURED

Organize so the model can navigate efficiently.

*An agent in a loop generates data that could matter next turn. Refine it **cyclically**, not once.*

Key takeaways

1. **Agents make context engineering critical.**
   Context flows from tool calls, not just humans — the #1 engineering job.

2. **Offload before you summarize.**
   Disk is your safety net. Summarization is lossy.

3. **Agentic retrieval beats heavyweight RAG (often).**
   Simple tools (grep, glob, fetch) with progressive disclosure win for many use cases.

4. **Read tasks parallelize, write tasks don't.**
   Sub-agents for research. Single-threaded for code generation.

UP NEXT: SESSION 3

Diagnosing & Fixing Context Failures

Four named failure modes, four matched fixes.

SESSION 3

Diagnosing and Fixing Context Failures

Identifying failure modes in long-context agents and applying targeted fixes.

The cost of unchecked context growth

- **Overflow** — agent hits the window limit and stops working
- **Cost explosion** — 500K tokens, $1–2 per naive deep-research run
- **Self-reinforcing loop** — each turn adds context the agent itself generated; errors compound

The agent literally feeds its own mistakes back into the next decision.

"When LLMs take a wrong turn in a conversation, they get lost and do not recover."

— Drew Breunig (citing Microsoft/Salesforce multi-turn study)

Four ways long context fails

POISONING

A hallucination enters context and compounds.

DISTRACTION

Context overwhelms training knowledge.

CONFUSION

Irrelevant content influences the response.

CLASH

Parts of context disagree with each other.

Drew Breunig's taxonomy. Each mode has distinct symptoms — and a distinct fix.

FAILURE 01

Context Poisoning

> *A hallucination makes it into the context, where it is repeatedly referenced.*

Gemini Plays Pokémon: the agent hallucinated game-state details, poisoned its goals section, and became **fixated on impossible goals**.

The agent treats its own bad output as ground truth. Errors don't get corrected — they amplify.

Fix: Isolate + Select

Primary lever: Isolate — keep raw tool outputs in separate state fields. Don't let them flow into history unchecked.

Then Select — promote only **validated** facts into the next prompt.

Supporting: Write a scratchpad of claims + sources. Only forward verified ones.

FAILURE 02

Context Distraction

> *Context grows so long that the model over-focuses on it and neglects what it learned in training.*

Gemini 2.5 Pro: decreased performance beyond **100K tokens** despite a 1M+ window. The agent defaulted to **repeating past actions** instead of reasoning fresh.

Llama 3.1 405B hit its distraction ceiling at ~32K tokens.

2026 — the ceiling moved, but didn't disappear. MRCR v2 (8-needle) at **1M tokens**: Claude Opus 4.6 **76%**, GPT-5.4 **36.6%**, Gemini 3 Pro **24.5%**. Nominal ≠ *usable* window.

Fix: Compress + Select

Primary lever: Compress — periodically summarize message history. Claude Code auto-compacts at 95%. Reinitiate with the summary.

Then Select — pull a smaller working set each step. Don't dump the full history every turn.

Supporting: Write a persistent plan/scratchpad so you can compress history without losing intent.

FAILURE 03

Context Confusion

> *Superfluous content influences the response. If you put something in the context, the model has to pay attention to it.*

Berkeley benchmark: Llama 3.1 8B **failed** on GeoEngine with 46 tools available — but **succeeded with only 19**.

Every model performs worse with more tools, even when those tools never get called.

Fix: Select + Isolate

Primary lever: Select — dynamic tool loading. LLM-powered tool recommender improved Llama 3.1 8B by **44%** on tool-selection tasks.

Then Isolate — keep "nice-to-have" context in separate fields. Inject only when required.

Supporting: Drew Breunig's "Tool Loadout" pattern — DeepSeek-v3 fails at 100+ tools; select a subset per task.

FAILURE 04

Context Clash

> *New information accumulates that conflicts with information already in context.*

Microsoft/Salesforce multi-turn study: 39% avg performance drop when identical info was spread across multiple turns vs given all at once.

Once the model takes a wrong turn, it **does not recover.**

Fix: Isolate + Select

Primary lever: Isolate — context quarantine. Separate threads with dedicated contexts. Anthropic's multi-agent system outperformed single-agent by **90.2%** on parallelizable tasks.

Then Select — apply precedence rules to decide what gets included this step.

Supporting: Compress conflicting sections into a single resolved summary after deciding precedence.

Failures → fixes at a glance

FAILURE MODE	PRIMARY FIX	SUPPORTING FIX	KEY SIGNAL
Poisoning	Isolate + Select	Write (claims+sources)	Bad fact propagates across steps
Distraction	Compress + Select	Write (persistent plan)	Agent repeats past actions
Confusion	Select + Isolate	Tool Loadout	Irrelevant tools/docs selected
Clash	Isolate + Select	Compress (resolved summary)	Flip-flopping between approaches

Diagnose first. Then pick the lever. Don't blindly add compression to everything.

Core principle: "Find the smallest set of high-signal tokens." — Anthropic

The debugging loop

1. **Inspect trace** — walk through messages, tool calls, tool outputs turn-by-turn
2. **Tag failure** — which of the four modes?
3. **Pick a lever** — W / S / C / I based on the mode
4. **Re-run & evaluate** — measure outcome quality, not just token count
5. **Iterate** — context engineering is cyclical, not one-shot

Measuring what matters

- **Track tokens per step** — find where context grew
- **Validate with evals** — token reduction without quality is a regression
- **Iterate** — pruning cut a Wikipedia article **95%** with QA accuracy preserved
- **Offload + domain prompts** — up to **54%** performance gain

Key takeaways

1. **Four failure modes, four names.**
   Poisoning, Distraction, Confusion, Clash. Each has different symptoms and different fixes.

2. **Diagnose before you fix.**
   Trace the trajectory, tag the mode, then pick the lever.

3. **Each failure maps to specific levers.**
   Select shows up in every fix. Compress is for distraction. Isolate is for poisoning and clash.

4. **Measure outcome quality, not token count.**
   Token reduction without eval improvement is a regression in disguise.

UP NEXT: SESSION 4

Context Engineering Patterns in Modern AI Apps

How production systems — Cursor, Manus, Vercel — apply these levers at scale.

SESSION 4

Context Engineering Patterns in Modern AI Apps

How production systems manage what enters the context window — and when.

The static-to-dynamic spectrum

STATIC

CLAUDE.md, .cursorrules. Always loaded. Predictable but stale.

TIERED

Hot/warm/cold layers loaded by role or trigger.

DYNAMIC

Agent discovers and fetches context on demand. Cursor, Claude Code.

SELF-IMPROVING

Contexts evolve from execution feedback. ACE-style.

Production systems combine multiple points on this spectrum.

Most CLAUDE.md files make agents worse.

ETH Zurich (arXiv 2602.11988, Feb 2026) evaluated AGENTS.md / CLAUDE.md files across 4 agents and 300+ tasks.

- LLM-generated files: **−3% success rate**, **+20% inference cost**
- Human-written files: **+4% marginal**, still costs more

Agents do **more** (27% more grep / read) without solving more tasks.

The paradox & what works

- **The paradox:** agents *do* follow CLAUDE.md — they use mentioned tools 1.6–2.5× more. But following costs 14–22% more reasoning tokens. **They work harder without solving better.**
- **What works:** human-written, **minimal**, only what the agent **genuinely can't infer** from the codebase (build commands, package manager quirks, non-obvious conventions).
- **Don't duplicate** information already in READMEs or code comments.

Context priming beats static dumping

At session start, ask the model to **find and load reference materials** with its own tools (web search, file read). The context becomes:

- **Accurate** — pulled fresh, not from stale training weights
- **Targeted** — specific to the current task
- **Always-fresh** — not always-on

Static = routing table. Dynamic = the actual content.

46.9%

Token reduction in runs that use MCP tools
after Cursor switched to **dynamic context discovery**.

Cursor Engineering · "Dynamic Context Discovery" · 2026

Production lessons from Manus

Instructor synthesis bridging two sources:

- **Breunig (2025)** named four failure modes. Two of them recur in production reports.
- **Manus / Peak Ji (2025)** described production degradation patterns:
  - "Context rot" — gradual decline as window fills → ≈ **Distraction**
  - "Context pollution" — noise drowns signal → ≈ **Confusion**

These are different vocabularies for the same phenomena.

Manus: five strategies for production

1. **Compaction & summarization** — reversible (paths, not contents) + lossy at ~128K tokens
2. **Communication over shared context** — fresh sub-agents for discrete tasks
3. **Hierarchical action spaces** — ~20 atomic tools; sandbox utilities; code libraries for complex chains
4. **Agent-as-tool** — sub-agents as deterministic functions with JSON schemas
5. **Pre-rot thresholds** — set compaction triggers at 70–80% utilization, **proactively**

> *"Share memory by communicating, don't communicate by sharing memory." — Peak Ji*

80% → 100%

80% → 100%

Vercel removed 80% of their text-to-SQL agent's tools.
Success rate went up. Tokens, steps, and latency went down.

Vercel Engineering · "We removed 80% of our agent's tools" · Dec 2025

Tool design = context engineering

ARCHITECTURE	TOKEN MULTIPLIER
Single agent (chat)	1× baseline
Single agent + tools	~4× baseline
Multi-agent system	~15× baseline

- **Consolidate** — fewer, more powerful tools beat many narrow ones
- **Tool descriptions are prompts** — vague params, missing error guidance = silent confusion
- **Tool outputs consume 83.9%** of total context in typical agent trajectories

Agent Skills: context loaded on demand

A skill is a folder: a `SKILL.md` with instructions, plus optional scripts and reference files.

STAGE	WHAT ENTERS THE CONTEXT
Always	Each skill's name + one-line description
When a task matches	The full `SKILL.md` body
Only if needed	Bundled scripts and reference files

Dozens of skills installed, almost none of them paying rent in the window.

Progressive disclosure — the Select lever, packaged.

Demo: a deck from data with the `pptx` skill

- **Input** — a sales CSV + a notes file, uploaded to the sandbox
- **No python-pptx instructions in the prompt** — just `skills: [pptx]` on the container
- **Claude reads `SKILL.md` on demand**, writes the deck, hands back a file

`demos/agents-with-skills/` · live-coded: `demos/live-agents-with-skills/`

Where agent performance actually lives

- **Token usage (context engineering)** — ~80% of performance variance
- **Tool calls** — ~10%
- **Model choice** — ~5%

Optimizing context has ~16× more impact than upgrading models.

Before going from Haiku to Opus, optimize what the agent sees. You'll likely get better results at lower cost.

Pattern selection guide

SITUATION	PATTERN	WHY
Stable project conventions	Minimal static file (CLAUDE.md)	Rarely changes
Topic-specific research	Context priming at session start	Fetches current docs
Large codebases, many tools	Dynamic discovery + tool consolidation	Avoids confusion
Long-running multi-step agents	Tiered architecture + filesystem context	Hot/cold separation
Multi-agent systems	Communication over shared context	Manus pattern
Repetitive learnable workflows	Self-improving contexts (ACE)	No manual maintenance

Key takeaways

1. **Static files help only when minimal and human-written.**
   ETH Zurich: LLM-generated CLAUDE.md hurts performance. Keep it short, keep it specific.

2. **Dynamic loading beats static dumping.**
   Cursor cut tokens 46.9% by letting the agent pull on demand.

3. **Fewer tools, better tools.**
   Vercel: 80% → 100% success after removing 80% of their tools.

4. **Context engineering is 16× more impactful than model choice.**
   Optimize what the agent sees before upgrading the model.

UP NEXT: SESSION 5

Tools and Techniques for Modern Development

Everything you've learned — applied with your day-to-day tools.

SESSION 5

Tools and Techniques for Modern Development

Hooks, sub-agents, MCP, scheduled agents, and the Claude Code primitives that make context engineering tractable.

The Claude Code primitives map

- `CLAUDE.md` — minimal static context (the routing table)
- **Custom slash commands** — repeatable workflows you author once
- **Hooks** — lifecycle events (`post-tool-use`, `pre-commit`) for sidecar files
- **Sub-agents** — isolated context windows, summary-only return
- **MCP servers** — bring your tools to the agent

Each primitive maps to one of the four levers from Session 1.

CLAUDE.md — keep it minimal

> *"Most CLAUDE.md files make agents worse." — ETH Zurich (arXiv 2602.11988, Feb 2026)*

What goes in:

- Project purpose (one line)
- Non-obvious commands (build, test, deploy)
- Hard-to-infer conventions

What stays out: anything already in the README. Anything the agent can `ls` to discover.

Custom slash commands

A custom slash command is a **reusable prompt** authored once, invoked from any project.

Patterns that work:

- `/standup` — draft standup notes (yesterday / today / blockers) from the git log
- `/ci_commit` — structured commit with conventional-commit prefix
- `/review` — diff-aware code review

Stored in `~/.claude/commands/` (user-global) or `.claude/commands/` (per-project).

Hooks: lifecycle events as a context lever

- `post-tool-use` — write tool output to a sidecar file (the Cursor "tool response files" pattern)
- `pre-commit` — gate context changes through validation
- `session-start` — auto-prime the context with current docs

Hooks are how you implement Compress + Isolate without changing your prompts.

Sub-agents in Claude Code

Each sub-agent gets:

- **Its own context window** (clean, no parent history)
- **Its own system prompt** (specialized role)
- **A scoped toolset** (only the tools it needs)

The parent agent sees only the **summary** the sub-agent returns. Token-heavy exploration stays isolated.

This is the Anthropic deep-research pattern, available as a built-in primitive.

MCP: bring your tools to the agent

Model Context Protocol = a standard way to expose tools, resources, and prompts to any MCP-aware agent.

- **Servers** publish capabilities
- **Clients** (Claude Code, Cursor, etc.) consume them
- One configuration → works across editors and CLIs

Composable context primitives, vendor-agnostic.

Scheduled agents: Claude Code on a cron

A cron entry runs a **headless** Claude Code agent. It does the work, then posts the result where you'll see it.

```
# Weekly dependency audit — every Sunday, 9am
0 9 * * 0  claude -p "Audit this repo's dependencies, flag anything outdated"
```

cron → `claude -p "<prompt>"` → Discord · file · vault · PR

What a scheduled agent can do

USE-CASE	WHAT IT DOES
Daily AI digest	Top stories summarized to Discord
Standup prep	Reviews git log, drafts yesterday / today / blockers
Weekly dependency audit	Flags outdated packages every Sunday
Nightly health check	Pings API, memory, model availability
Apple Notes → Obsidian	Summarizes notes, compiles into vault

Each row is one cron-driven Claude Code run.

Why context engineering makes unattended runs reliable

No human is there to fix a bloated or poisoned context mid-run.

- **Tool loadout** — only the tools the job needs (`--allowedTools`)
- **Fresh context per run** — every invocation starts clean
- **Grounding over memory** — `Read` the git log, package file, notes
- **Structured hand-off** — end with a well-formed post, not a transcript

Use-cases and cron entries: `demos/ctx-engineering-tools-claude-code/`

The cheatsheet

`assets/context-engineering-tools-cheatsheet.pdf` — every pattern from the five sessions on a printable four-page reference.

- W / S / C / I lever cheatsheet
- Four failure modes + matched fixes
- Pattern selection guide
- Claude Code primitive → lever mapping

This is the artifact you keep open while building your next agent.

Build your own playbook

You have:

- A **vocabulary** — Write / Select / Compress / Isolate
- A **taxonomy** — Poisoning / Distraction / Confusion / Clash
- A **toolkit** — Claude Code primitives + the cheatsheet
- A **discipline** — diagnose, lever, measure, iterate

Now: go apply it to one real agent at work this week.

Questions?

Repo · slides · cheatsheet · demos:

github.com/EnkrateiaLucca/context-engineering-hands-on